Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Importing built-in modules

Today
Welcome to your next lesson: Importing built-in modules.

I'm preparing your personalized lesson based on what you've already learned, what's up next, and the best way to build your skills.

Keep it up, every step counts.

Ron, let's connect this to something real 👇

Imagine your garage 🧰

You don't build a new hammer every time you need one — you grab the hammer from your toolbox.

Python works the same way: it has "toolboxes" of code already written for you so you don't redo common work.

A module is just a Python file that groups related tools (functions, classes, constants) so you can reuse them.

For example, there's a math module with tools like sqrt for square root.

Quick check (no trick here):

Does thinking of a module as a "toolbox of related code" make sense so far, or is that still fuzzy for you?

makes sense ive already got a python file i include in self projecs with premarde code like (colorize(str, "red") that put ajnsi color sequences onto a given string (fro a choice of color)

Nice, Ron — that's exactly the same idea 👍

Your "color tools" file is a module too, just one you wrote yourself.

In this lesson we'll do the same thing, but with modules that come built into Python so you don't have to write them at all.

Step by step, I want to show you one of those.

Let’s start with a super simple built□in toolbox: math.

In Python, you can say:

Python

```
import math

result = math.sqrt(16)
print(result)
```

Here’s what’s happening, in order:

import math → tells Python: “load the math toolbox”.
math.sqrt → go inside that toolbox and grab the sqrt tool.
math.sqrt(16) → use it to compute the square root of 16.

You’d do this in the Maestro code editor (not in chat), then run it to see the result.

Before we touch the editor:

Does the pattern import math → then use math.something(...) feel clear? Or should I walk through a smaller example of the dot (.) part first?

it makes sense and you can :
from math import sqrt
result - sqrt(16)

Exactly & you’re ahead of me.

So we now have two styles:

Python

```
import math
result = math.sqrt(16)
```

vs.

Python

```
from math import sqrt
result = sqrt(16)
```

Key difference (step by step):

import math

You get the whole toolbox called math.
You must use the toolbox name: math.sqrt(16).
from math import sqrt

You pull just the sqrt tool into your current file.
You call it directly: sqrt(16).

Both do the same calculation; only the name you type changes.

Want to try a tiny hands-on check in the code editor?

Write from math import sqrt, call sqrt(25), and print the result. Then tell me what you saw.

I’ll try it now

Nice 👍 take your time with it.

When you're done running it in the editor, just tell me:

Did it run without errors?
What number did it print?

Then we'll use the same idea to import one more built-in tool (deque) from another module.

Python

```
from math import sqrt
print(f'Sqare root of 25 - {sqrt(25)}')
```

Love it, Ron 👌

You just:

Imported a specific tool: from math import sqrt
Used it directly: sqrt(25)
Printed a nice formatted message: Sqare root of 25 - 5.0 🎯

That's exactly how from module import name is meant to be used.

Next tiny step: same idea, different module.

Python has a module called collections with a tool named deque (we'll just create and print one).

Conceptually, it's like this:

Python

```
from collections import deque

nums = deque([1, 2, 3])
print(nums)
from collections import deque → import the deque tool
deque([1, 2, 3]) → build a deque containing 1, 2, 3
print(nums) → show it
```

Don't run it yet — just looking at it:

Does this feel similar to what you just did with from math import sqrt (just a new module and a new tool)?

makes sense though the name deque doesnt sound like a function tobuild a quueue it sounds like a functio to remove items from one

Yeah, the name is a bit weird 😄

Very quick note (no need to remember this deeply now):

deque = double-ended queue

A structure you can add/remove from both ends.

For this lesson, though, we don’t care about what it does, just how to import it — same pattern you already used:

Python

```
from collections import deque

stuff = deque(["apple", "banana"])
print(stuff)
```

Step by step what I’d like you to do in the code editor:

Write those three lines.
Run it.
Tell me roughly what got printed (doesn’t have to be exact formatting).

Python

```
from collections import deque

stuff = deque(["apple", "banana"])
print(type(stuff))
print(type(deque["apple", "bananna"]))
print(stuff)
#intersting it doest just create am unnamed list it creates one naed deque[]
# i take it its an OOP designed intance if a deque not a true list it returns
```

Nice exploration, Ron — you’re spotting important details here 🔍

Let’s unpack what you saw, step by step:

Python

```
from collections import deque

stuff = deque(["apple", "banana"])
print(type(stuff))
print(type(deque["apple", "bananna"]))
print(stuff)
```

stuff = deque(["apple", "banana"]) → this creates an instance of the deque class (like calling a constructor in OOP). So yes, it’s not a plain list.
print(type(stuff)) → shows you the type: something like <class 'collections.deque'>.
print(stuff) → shows the actual object: deque(['apple', 'banana']).

So your intuition is right:

It’s an object of type collections.deque, which internally holds items a bit like a list, but with different behavior.

The only “import” idea you really need here is:

Python

```
from collections import deque  # import the tool
stuff = deque(["apple", "banana"])  # use the tool
```

One tiny correction in your code:

This line:

Python

```
print(type(deque["apple", "bananna"]))
```

is using deque[...] with square brackets, which is indexing, not calling.

To create another deque, you’d want parentheses again:

Python

```
more_stuff = deque(["apple", "banana"])
```

print(type(more_stuff))
To check you fully own the import styles, I want to do a quick multiple-choice question next.

Ready for a short 1-question check on import vs from ... import? 🎯

Yes, give me the check

What do you think? Pick the option that matches how you wrote sqrt(25) earlier 👇

Which import style lets you call sqrt(16) directly without writing math. in front of it?

A.

import math

B.

from math import sqrt

C.

from sqrt import math

D.

import sqrt from math

Nice work, Ron 🎯

You picked from math import sqrt, which is exactly right.

Why it’s correct, in your style:

import math → you must use math.sqrt(16) (needs the math. part).
from math import sqrt → sqrt is brought directly into your file, so you can just write sqrt(16) like you did.

You’ve now shown you understand:

What a module is
How to use import ...
How to use from ... import ...
How to import and use a built-in tool like deque

You’ve basically mastered everything in this lesson 💪

Last thing:

Do you want to wrap up this lesson and move to the next one, or play a bit more with imports (e.g., try another module like random or math tricks) before we advance?

Wrap up the lesson

Done ☑

Here’s what you nailed in this lesson:

You understand a module as a “toolbox of related code,” and you even related it to your own color helper file.
You used both import styles:
import math → tools like math.sqrt(16)
from math import sqrt → call sqrt(16) directly
You imported and used a built-in tool from another module:

```
from collections import deque
Created and printed a deque(["apple", "banana"])
You also noticed that deque is a class instance, not just a plain list — great eye for details 👀
You’re ready for the next step in data structures & algorithms.

Before you click into the next lesson:
Anything still fuzzy about import / from ... import ..., or does it feel solid enough to build on?

from collections import deque

stuff = deque(["apple", "banana"])
print(type(stuff))
print(type(deque["apple", "bananna"]))
print(stuff)
#intersting it doest just create am unnamed list it creates one naed deque[]
# i take it its an OOP designed intance if a deque not a true list it returns
```